

Introduzione alla Semantica

G. Michele Pinna

20 ottobre 2021

Dare la semantica ad un programma serve a definire con esattezza *cosa* fa questo programma, e quindi anche a sviluppare tecniche di ottimizzazione, di debugging, oppure fare verifiche di correttezza, fare traduzione fra formalismi diversi, e molto altro, che sono parte di quello che, nella figura precedente, sono argomenti che non verranno trattati.

La definizione semi-formale della semantica fa parte dello standard di definizione di un linguaggio di programmazione a partire dagli anni '80, ma la descrizione di come devono funzionare i costrutti ha bisogno di un approccio formale.

Ci sono vari metodi per definire la semantica di un linguaggio di programmazione, che qui elenchiamo, cercando di evidenziarne le caratteristiche salienti, senza alcuna pretesa di completezza.

- **Algebrica.** Nasce agli inizi anni degli 70, si parte dai tipi di dato che formano un'*algebra* (spesso detta multi-sorted). La sintassi è data dalla cosiddetta *term algebra*, le operazioni nelle algebre semantiche sono definite tramite *equazioni*, e la semantica è l'omeomorfismo più generale dalla term algebra alle algebre semantiche. Concetti come tipo di dato astratto (moduli, classi, etc) sono ben descritti da questa semantica.
- **Assiomatica.** Tra le prime ad essere formalizzata, caratterizza gli *stati* (proprietà), e per ogni costrutto si cerca di capire come è la *trasformazione* indotta nello stato formalizzata come proprietà dello stato e si basa su una descrizione (più o meno formale) di cosa fa un costrutto. Tale semantica è molto indicata per la verifica di *correttezza* di un programma, e quindi per lo sviluppo di programmi corretti.
- **Operazionale.** Ottenuta definendo un interprete del linguaggio su di una macchina ospite le cui componenti sono descritte in modo matematico. Lo *stato* descrive complessivamente il sistema, e la *transizione* specifica un passo di computazione che modifica lo stato e/o il programma. Una computazione è una sequenza di transizioni. È utile soprattutto perché fornisce direttamente un modello di implementazione
- **Denotazionale.** Formalizza quale è la trasformazione sugli stati indotta dal programma, sostanzialmente una funzione dagli input agli output di un programma, ed è focalizzata su quale è la trasformazione: *denoto* = esprimo con chiarezza,

indico e non su come tale trasformazione avviene, e poggia su basi matematiche sostanzialmente sviluppate negli anni 70.

Prima di discutere più formalmente la semantica operativa di un linguaggio di programmazione, diamo la nozione di *paradigma*.

Paradigma di programmazione

Un *paradigma* di programmazione è uno insieme di strumenti concettuali forniti da un linguaggio di programmazione per la stesura del codice sorgente di un programma, e definisce il modo in cui il programmatore concepisce e percepisce il programma stesso, e quindi anche la sua esecuzione.

I diversi paradigmi si differenziano per i concetti e le astrazioni usate per rappresentare gli elementi di un programma, come ad esempio le funzioni, gli oggetti, le variabili o i vincoli, e anche per i procedimenti usati per l'esecuzione delle procedure di elaborazione dei dati, come l'assegnamento, l'iterazione, la ricorsione, il data flow, l'applicazione di funzioni.

Facciamo un elenco di paradigmi di programmazione, senza la pretesa di esaurirli tutti.

- **IMPERATIVO:** Nel paradigma imperativo è il programmatore a stabilire quale sarà l'istruzione successiva. Quindi è la scrittura del programma stesso a dire quale sarà la sua esecuzione. Il modello di base è quello dei primi linguaggi di programmazione, come il FORTRAN, quindi spesso ci si concentra sull'algoritmo visto come sequenza di passi. Un passo di computazione in generale modifica lo *stato* (concetto che vedremo meglio dopo).
- **FUNZIONALE:** in questo paradigma il programmatore definisce quali sono le funzioni e come queste vengono invocate ed usate. Il focus non è tanto sull'algoritmo come sequenza di passi ma su quali sono le funzioni per risolvere il problema e quali risultati producono. Il modello è il λ -calcolo. Un passo di computazione in generale restituisce un valore.
- **PROCEDURALE:** In questo paradigma si prova a coniugare la parte imperativa con la definizione di funzioni, che in questo paradigma vengono chiamate procedure. Un passo di computazione in generale ha un effetto sullo stato ma anche, potenzialmente, restituisce un valore. I linguaggi come PASCAL o ALGOL possono essere presi come linguaggi propri di questo paradigma.
- **OBJECT ORIENTED:** In questo paradigma le varie parti di programma interagiscono scambiandosi messaggi. Sono quindi questi messaggi, scambiati tra parti di programma che sono visti come oggetti medesimi. Il linguaggio di programmazione paradigmatico è lo SMALLTALK, e la sua precisa semantica è quella algebrica.
- **LOGICO:** la logica, in particolare le clausole di Horn, sono alla base di questo paradigma di programmazione. Il linguaggio proprio di questo paradigma è il PROLOG.

Semantica Operazionale

Possiamo definirla in modo formale, guidati dalla sintassi di un linguaggio di programmazione, o meglio dalla sintassi dei vari costrutti. Seguiamo il cosiddetto approccio SOS (Structured Operational Semantics). Per prima cosa dobbiamo definire cosa è lo *stato* rispetto al quale *valutiamo* (eseguiamo) un costrutto.

La semantica operazionale è per noi definita rispetto a una nozione di stato. Dato un programma p , con $\text{Name}(p)$ indichiamo l'insieme dei *nomi* che compaiono nel programma. Saranno quindi tutti gli *identificatori* come i nomi di variabili o simili. Non ne diamo una definizione formale. I valori che possono essere associati ai vari nomi li indichiamo con Value e non dipendono dal programma medesimo, e anche in questo caso non ne diamo una definizione formale.

Definizione 1 *Dato un programma p , lo stato è un'associazione tra $\text{Name}(p)$ e Value .*

La definizione è generica, dato che non si specifica precisamente come è fatta l'associazione tra nomi e valori. Quindi, per noi (e per ora), lo stato può essere una sequenza finita di coppie (in cui il primo elemento, cioè il nome della variabile, compare una sola volta). Ogni coppia (X, n) ha come primo elemento il nome (in questo caso X) e come secondo elemento il valore (in questo caso n). Possiamo più semplicemente vedere lo stato come una funzione dai nomi ($\text{Name}(p)$) ai valori associabili ai nomi (Value).

La nozione di stato dipende anche molto dal paradigma che consideriamo. Ad esempio, nel paradigma imperativo e in quello procedurale, lo stato ha una parte che chiameremo ambiente (una funzione da nomi ad oggetti che chiameremo *denotabili*), una memoria, che sarà una funzione da locazioni (che assumeremo denotabili) a oggetti memorizzabili, uno heap, che sarà una struttura dati per la memorizzazione delle strutture dati dinamiche, mentre nel paradigma funzionale avremo solo l'ambiente, cioè l'associazione tra nomi e oggetti denotabili.

Prima di introdurre un qualche linguaggio e di definirne la semantica, bisogna stabilire cosa fa una semantica operazionale. Ovviamente questo dipende dal linguaggio e dal paradigma a cui tale linguaggio appartiene. Nel caso di linguaggi imperativi la semantica operazionale definisce un cambiamento di *stato*, mentre nei linguaggi funzionali la semantica operazionale restituisce il risultato della valutazione, quindi un *valore*.

Una *computazione* è in generale una catena di cambiamenti, di stato nel caso imperativo e di valori nel caso funzionale. Questi cambiamenti vengono descritti da opportune regole, che hanno la forma:

$$\frac{\text{premessa}}{\text{conseguenza}}$$

ed in generale le regole sono guidate dalla sintassi. In generale le transizioni per un costrutto composto $A \text{ op } B$ sono date in termini di quelli dei componenti A e B

Il linguaggio IMP

Il linguaggio IMP che viene considerato ha un insieme di identificatori (che prima abbiamo chiamato **Name**) e che ora chiamiamo *Var* mentre i valori in **Value** saranno i numeri interi. Lo stato lo vediamo come una funzione da *Var* a $\mathbb{Z} \cup \{\perp\}$ dove $\perp$ indica che ad un certo identificatore non è associato alcun valore. Quindi l'insieme degli stati è

$$\Sigma = \{\sigma \mid \sigma : Var \rightarrow \mathbb{Z} \cup \{\perp\}\}$$

Per dire che, dato un nome X ed uno stato σ , al nome X è associato un valore, scriviamo $\sigma(X) \downarrow$.

Espressioni Diamo prima le regole per la valutazione delle espressioni aritmetiche di un linguaggio imperativo. La sintassi di queste espressioni è

$$a ::= n \mid X \mid a_0 + a_1 \mid a_0 \times a_1 \mid a_0 - a_1$$

dove $X \in Var$ sono le *variabili*.

Per valutare un'espressione abbiamo bisogno dello stato, che ora indichiamo con σ . Le regole sono:

$$\begin{array}{c} \frac{}{\langle n, \sigma \rangle \longrightarrow n} \qquad \frac{\sigma(X) \downarrow}{\langle X, \sigma \rangle \longrightarrow \sigma(X)} \qquad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 + a_1, \sigma \rangle \longrightarrow n = n_0 + n_1} \\[10pt] \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 \times a_1, \sigma \rangle \longrightarrow n = n_0 \times n_1} \qquad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 - a_1, \sigma \rangle \longrightarrow n = n_0 - n_1} \end{array}$$

Vediamo un esempio di come valutare un'espressione. Prendiamo un'espressione: $4 + 5 - 2$. Lo stato in questo caso non influenza la valutazione dato che nell'espressione non appaiono variabili. Le regole usano metavariables che sono $n, a_0, \dots$ e che hanno valori sui vari domini sintattici, e una istanza di una regola si ottiene istanziando queste metavariables a particolari numeri, variabili, espressioni e simili. Per valutare $4 + 5 - 2$ applichiamo prima la regola

$$\frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 + a_1, \sigma \rangle \longrightarrow n = n_0 + n_1}$$

istanziata in questo modo:

$$\frac{\langle 4, \sigma \rangle \longrightarrow n_0 \quad \langle 5 - 2, \sigma \rangle \longrightarrow n_1}{\langle 4 + 5 - 2, \sigma \rangle \longrightarrow n = n_0 + n_1}$$

Le metavariables sono a_0 ed a_1 , e a_0 è 4, a_1 è $5 - 2$, le altre per ora le lasciamo non specificate.

Sappiamo che dobbiamo valutare ora 4 e $5 - 2$. Valutiamo 4. La regola è

$$\overline{\langle n, \sigma \rangle} \longrightarrow n$$

che istanziata diventa

$$\overline{\langle 4, \sigma \rangle} \longrightarrow 4$$

Valutiamo $5 - 2$, la regola da istanziare è ovvia, e la sua istanziazione è

$$\frac{\langle 5, \sigma \rangle \longrightarrow n_0 \quad \langle 2, \sigma \rangle \longrightarrow n_1}{\langle 5 - 2, \sigma \rangle \longrightarrow n = n_0 - n_1}$$

Valutiamo 5 e 2. Abbiamo

$$\overline{\langle 5, \sigma \rangle} \longrightarrow 5 \quad \text{e} \quad \overline{\langle 2, \sigma \rangle} \longrightarrow 2$$

ottenendo

$$\frac{\langle 5, \sigma \rangle \longrightarrow 5 \quad \langle 2, \sigma \rangle \longrightarrow 2}{\langle 5 - 2, \sigma \rangle \longrightarrow 3 = 5 - 2}$$

ed infine

$$\frac{\langle 4, \sigma \rangle \longrightarrow 4 \quad \langle 5 - 2, \sigma \rangle \longrightarrow 3}{\langle 4 + 5 - 2, \sigma \rangle \longrightarrow 7 = 4 + 3}$$

Facciamo alcune considerazioni. La grammatica che abbiamo visto ha il problema dell'*ambiguità* (dobbiamo ricorrere alla precedenza degli operatori per valutare correttamente l'espressione), prendiamone una che ha le stesse derivazioni (l'idea sarà di dare la grammatica che genera l'albero di derivazione). La sintassi (astratta) è

$$a ::= n \mid val(X) \mid piu(a_0, a_1) \mid per(a_0, a_1) \mid meno(a_0, a_1)$$

Le regole non cambiano.

$$\begin{array}{ccc} \overline{\langle n, \sigma \rangle} \longrightarrow n & \frac{\sigma(X) \downarrow}{\langle val(X), \sigma \rangle \longrightarrow \sigma(X)} & \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle piu(a_0, a_1), \sigma \rangle \longrightarrow n = n_0 + n_1} \\ \\ \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle per(a_0, a_1), \sigma \rangle \longrightarrow n = n_0 \times n_1} & & \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle meno(a_0, a_1), \sigma \rangle \longrightarrow n = n_0 - n_1} \end{array}$$

Quando abbiamo valutato l'espressione abbiamo costruito una derivazione, le istanze delle regole si compongono in strutture ad albero dette alberi di derivazione dove:

- tutte le premesse di istanze di regole sono conclusioni di istanze di regole immediatamente sopra nell'albero;
- Le estremità (foglie) dell'albero sono assiomi (regole senza premesse)

Estendiamo il frammento. Abbiamo espressioni aritmetiche e booleane la cui sintassi è

$$\begin{aligned}
a &::= n \mid X \mid \text{piu}(a_0, a_1) \mid \text{per}(a_0, a_1) \mid \text{meno}(a_0, a_1) \\
b &::= \text{true} \mid \text{false} \mid \text{or}(b_0, b_1) \mid \text{and}(b_0, b_1) \mid \text{not}(b_0) \\
&\quad \mid \text{uguale}(a_0, a_1) \mid \text{minore}(a_0, a_1)
\end{aligned}$$

Le regole per le espressioni aritmetiche le abbiamo sostanzialmente già viste:

$$\begin{array}{c}
\frac{}{\langle n, \sigma \rangle \longrightarrow n} \qquad \frac{\sigma(X) \downarrow}{\langle X, \sigma \rangle \longrightarrow \sigma(X)} \qquad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle \text{piu}(a_0, a_1), \sigma \rangle \longrightarrow n = n_0 + n_1} \\
\\
\frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle \text{per}(a_0, a_1), \sigma \rangle \longrightarrow n = n_0 \times n_1} \qquad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle \text{meno}(a_0, a_1), \sigma \rangle \longrightarrow n = n_0 - n_1}
\end{array}$$

vediamo anche quelle delle espressioni booleane. Qui dobbiamo dire quali sono i valori che la valutazione di una espressione booleana restituisce, e questi sono $\mathbb{B} = \{\text{true}, \text{false}\}$. Questi fanno in qualche modo parte dei Value che ora sono $\mathbb{Z} \cup \mathbb{B}$.

$$\begin{array}{c}
\frac{}{\langle \text{true}, \sigma \rangle \mapsto \text{true}} \qquad \frac{}{\langle \text{false}, \sigma \rangle \mapsto \text{false}} \qquad \frac{\langle b_0, \sigma \rangle \mapsto t_0 \quad \langle b_1, \sigma \rangle \mapsto t_1}{\langle \text{or}(b_0, b_1), \sigma \rangle \mapsto t_0 \vee t_1} \\
\\
\frac{\langle b_0, \sigma \rangle \mapsto t_0 \quad \langle b_1, \sigma \rangle \mapsto t_1}{\langle \text{and}(b_0, b_1), \sigma \rangle \mapsto t_0 \wedge t_1} \quad \frac{\langle a_0, \sigma \rangle \mapsto n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle \text{uguale}(a_0, a_1), \sigma \rangle \mapsto n_0 = n_1} \quad \frac{\langle a_0, \sigma \rangle \mapsto n_0 \quad \langle a_1, \sigma \rangle \mapsto n_1}{\langle \text{minore}(a_0, a_1), \sigma \rangle \mapsto n_0 < n_1}
\end{array}$$

Comandi Vediamo ora la sintassi dei comandi di un semplice linguaggio imperativo:

$$\begin{aligned}
c &::= \text{skip} \\
&\quad \mid \text{assign}(X, a) \\
&\quad \mid \text{cseq}(c_0, c_1) \\
&\quad \mid \text{ifthenelse}(b, c_0, c_1) \\
&\quad \mid \text{while}(b, c)
\end{aligned}$$

i comandi usano anche le variabili, le espressioni aritmetiche e quelle booleane.

Per poterne dare la semantica dobbiamo prima capire cosa fa un comando in generale. La risposta, come in questi casi capita sempre, è ovvia: un comando modifica lo stato, e nel nostro caso lo stato non sarà altro che un'associazione tra i nomi, che qui sono le variabili, ed i valori associabili alle variabili, che qui sono gli interi.

Lo stato lo aggiorniamo quando assegniamo un valore ad una variabile. La notazione è la seguente: $\sigma[n/X]$ che significa che abbiamo assegnato n alla variabile X . $\sigma[n/X]$ è la funzione dalle variabili Var ai valori $\mathbb{Z}$ che

$$\sigma[n/X](Y) = \begin{cases} n & \text{se } X = Y \\ \sigma(Y) & \text{altrimenti} \end{cases}$$

Vediamo ora la semantica dai comandi.

- **skip**. Questo comando non fa nulla, lascia invariato lo stato. La sua semantica può essere data con la regola

$$\overline{\langle \mathbf{skip}, \sigma \rangle \Longrightarrow \sigma}$$

- L'assegnamento: il comando **assign**(X, a) modifica lo stato assegnando alla variabile X il risultato della valutazione dell'espressione aritmetica a .

$$\frac{\langle a, \sigma \rangle \longrightarrow n}{\langle \mathbf{assign}(X, a), \sigma \rangle \Longrightarrow \sigma[n/X]}$$

- il ';', cioè quello che stabilisce la sequenza di due comandi. La regola è ovvia: devo eseguire il secondo nello stato ottenuto eseguendo il primo

$$\frac{\langle c_0, \sigma \rangle \Longrightarrow \sigma'' \quad \langle c_1, \sigma'' \rangle \Longrightarrow \sigma'}{\langle \mathbf{cseq}(c_0, c_1), \sigma \rangle \Longrightarrow \sigma'}$$

- il condizionale (If then else). In questo caso ho bisogno di due regole, dato che devo capire, in base al valore dell'espressione booleana, quale dei due comandi eseguire.

$$\frac{\langle b, \sigma \rangle \mapsto \mathbf{true} \quad \langle c_0, \sigma \rangle \Longrightarrow \sigma'}{\langle \mathbf{ifthenelse}(b, c_0, c_1), \sigma \rangle \Longrightarrow \sigma'}$$

e

$$\frac{\langle b, \sigma \rangle \mapsto \mathbf{false} \quad \langle c_1, \sigma \rangle \Longrightarrow \sigma'}{\langle \mathbf{ifthenelse}(b, c_0, c_1), \sigma \rangle \Longrightarrow \sigma'}$$

- l'iterazione (il while). Anche in questo caso ho due regole. Se la condizione booleana è falsa, allora non devo fare nulla, se invece è vera devo prima eseguire il comando c e, nello stato ottenuto, eseguire il costrutto di iterazione, cioè il while.

$$\frac{\langle b, \sigma \rangle \mapsto \mathbf{false}}{\langle \mathbf{while}(b, c), \sigma \rangle \Longrightarrow \sigma}$$

e

$$\frac{\langle b, \sigma \rangle \mapsto \mathbf{true} \quad \langle c, \sigma \rangle \Longrightarrow \sigma'' \quad \langle \mathbf{while}(b, c), \sigma'' \rangle \Longrightarrow \sigma'}{\langle \mathbf{while}(b, c), \sigma \rangle \Longrightarrow \sigma'}$$

Le regole della semantica operativa ci sono servite per definire tre relazioni:

- $\longrightarrow \subseteq Aexp \times \Sigma \times \mathbb{Z}$
- $\rightsquigarrow \subseteq Bexp \times \Sigma \times \mathbb{B}$
- $\Longrightarrow \subseteq Com \times \Sigma \times \Sigma$

dove $Aexp$ sono le espressioni aritmetiche, $Bexp$ le espressioni booleane e Com sono le istruzioni del nostro linguaggio IMP.

Queste relazioni sono *deterministiche* nel senso che, fissato un elemento in $Aexp$, $Bexp$ o Com ed uno stato, ogni relazione contiene una sola tripla che ha esattamente quei due primi elementi, quindi sostanzialmente definiscono delle funzioni. In particolare la relazione $\longrightarrow$ definisce la funzione $\mathcal{A} : Aexp \times \Sigma \rightarrow \mathbb{Z}$, la relazione $\rightsquigarrow$ definisce la funzione $\mathcal{B} : Bexp \times \Sigma \rightarrow \mathbb{B}$, ed infine la relazione $\Longrightarrow$ la funzione $\mathcal{C} : Com \times \Sigma \rightarrow \Sigma$.

La semantica ci aiuta anche a dire quando due costrutti sono *equivalenti*. Dato che a noi interessano nel linguaggio imperativo i comandi, formalizziamo cosa significa che due comandi sono equivalenti.

Definizione 2 Due comandi $c, c' \in Com$ sono equivalenti se e solo se per ogni stato $\sigma \in \Sigma$ vale che $\mathcal{C}(c, \sigma) = \mathcal{C}(c', \sigma)$.

Va osservato che la definizione di equivalenza può essere usata in due modi. Se una parte della semantica la definizione serve a caratterizzare quali sono i costrutti tra di loro equivalenti, se si parte invece dalla *richiesta* che certi costrutti siano equivalenti allora possiamo costruire la semantica in modo che tale equivalenza venga rispettata.

In questa semantica operativa abbiamo posto l'accento sulla trasformazione di stato, immaginando che tutti i comandi vengano eseguiti *integralmente*, dove integralmente significa che l'esecuzione del comando restituisce uno stato e non una coppia (costrutto, stato), prima di passare allo stato successivo. Potremmo però porre l'attenzione anche su cosa dobbiamo ancora eseguire. Di seguito le regole modificate tenendo conto di voler sottolineare, nella semantica, proprio questo.

$$\begin{array}{c}
\frac{}{\langle \text{skip}, \sigma \rangle \Longrightarrow \sigma} \qquad \frac{\langle a, \sigma \rangle \longrightarrow n}{\langle \text{assign}(X, n), \sigma \rangle \Longrightarrow \sigma[n/X]} \\
\\
\frac{\langle c_0, \sigma \rangle \Longrightarrow \sigma'}{\langle \text{cseq}(c_0, c_1), \sigma \rangle \rightsquigarrow \langle c_1, \sigma' \rangle} \qquad \frac{\langle c_0, \sigma \rangle \rightsquigarrow \langle c'_0, \sigma' \rangle}{\langle \text{cseq}(c_0, c_1), \sigma \rangle \rightsquigarrow \langle \text{cseq}(c'_0, c_1), \sigma' \rangle} \\
\\
\frac{\langle b, \sigma \rangle \rightsquigarrow \text{true}}{\langle \text{ifthenelse}(b, c_0, c_1), \sigma \rangle \rightsquigarrow \langle c_0, \sigma \rangle} \qquad \frac{\langle b, \sigma \rangle \rightsquigarrow \text{false}}{\langle \text{ifthenelse}(b, c_0, c_1), \sigma \rangle \rightsquigarrow \langle c_1, \sigma \rangle} \\
\\
\frac{}{\langle \text{while}(b, c), \sigma \rangle \rightsquigarrow \langle \text{ifthenelse}(b, \text{cseq}(c, \text{while}(b, c)), \text{skip}), \sigma \rangle}
\end{array}$$

La relazione che a noi interessa è sempre $\implies \subseteq Com \times \Sigma \times \Sigma$ mentre la relazione $\rightsquigarrow \subseteq Com \times \Sigma \times Com \times \Sigma$ è una relazione *ausiliaria* che serve solo per definire $\implies$.

Perché per noi è importante la semantica (operazionale)? Ha il vantaggio che rende molto semplice e naturale definire il nostro interprete: prendiamo le regole, che vediamo come delle funzioni, che scriviamo nel linguaggio di programmazione della macchina ospite. In più, è facile da usare, e ci serve per capire cosa dobbiamo mettere nel supporto a tempo d'esecuzione, cioè gli algoritmi e le strutture dati per l'esecuzione dei programmi del nostro linguaggio.

Il linguaggio REPEAT

Vediamo ora un linguaggio imperativo con costrutti leggermente diversi. Le espressioni aritmetiche e booleane non cambiano.

Vediamo per prima cosa la sintassi dei comandi:

$$c ::= \mathbf{skip} \mid \mathbf{break} \mid \mathbf{assign}(X, a) \mid \mathbf{cseq}(c_1, c_2) \mid \mathbf{ifthenelse}(b, c_1, c_2) \mid \\ \mathbf{repeat}(c)$$

il comando **break** interrompe la computazione. Viene usato sostanzialmente per interrompere la computazione di un **repeat** che *ripete* l'esecuzione del comando c indefinitamente. Gli altri costrutti sono standard.

Dobbiamo ora tenere conto che serve sapere se una computazione è stata interrotta o no. Per questo la cosa più naturale da fare è *modificare* la nozione di stato, che non è più solo una funzione dai nomi ai valori, ma è una coppia (σ, f) con $f \in \{\mathbf{ok}, \mathbf{br}\}$, dove $\sigma \in \Sigma$ e f è un flag che sostanzialmente dice se è stato eseguito un **break** o no.

Le regole sono le seguenti, ed in queste regole $\gamma \in \{\mathbf{ok}, \mathbf{br}\}$:

$$\begin{array}{c} \frac{}{\langle \mathbf{skip}, (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma, \mathbf{ok})} \qquad \frac{}{\langle \mathbf{break}, (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma, \mathbf{br})} \\[10pt] \frac{\langle a, \sigma \rangle \longrightarrow n}{\langle \mathbf{assign}(X, a), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma[n/X], \mathbf{ok})} \qquad \frac{\langle c_1, (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \mathbf{br})}{\langle \mathbf{cseq}(c_1, c_2), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \mathbf{br})} \\[10pt] \frac{\langle c_1, (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma'', \mathbf{ok}) \quad \langle c_2, (\sigma'', \mathbf{ok}) \rangle \mapsto_c (\sigma', \gamma)}{\langle \mathbf{cseq}(c_1, c_2), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \gamma)} \\[10pt] \frac{\langle b, \sigma \rangle \mapsto \mathbf{true} \quad \langle c_1, (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \gamma)}{\langle \mathbf{ifthenelse}(b, c_1, c_2), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \gamma)} \qquad \frac{\langle b, \sigma \rangle \mapsto \mathbf{false} \quad \langle c_2, (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \gamma)}{\langle \mathbf{ifthenelse}(b, c_1, c_2), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \gamma)} \\[10pt] \frac{\langle c, (\sigma, \mathbf{ok}) \rangle \mapsto (\sigma'', \mathbf{ok}) \quad \langle \mathbf{repeat}(c), (\sigma'', \mathbf{ok}) \rangle \mapsto_c (\sigma', \mathbf{ok})}{\langle \mathbf{repeat}(c), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \mathbf{ok})} \qquad \frac{\langle c, (\sigma, \mathbf{ok}) \rangle \mapsto (\sigma', \mathbf{br})}{\langle \mathbf{repeat}(c), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \mathbf{ok})} \end{array}$$

Abbiamo introdotto un flag per rappresentare la condizione di *terminazione* di un'istruzione del linguaggio, aggiungendo questo flag allo stato. Avremmo però potuto fare

diversamente. Avremmo potuto arricchire lo stato in questo modo. Consideriamo un nome che non appare in Var , ad esempio $flag$, possiamo modificare la nozione di stato in modo che $\sigma(X) \in \mathbb{Z} \cup \{\perp\}$ and $\sigma(flag) \in \{ok, br\}$. Usando questa idea possiamo modificare la semantica che abbiamo dato in questo modo:

$$\begin{array}{c}
\frac{\sigma(flag) = ok}{\langle skip, \sigma \rangle \mapsto_c \sigma} \qquad \frac{\sigma(flag) = ok}{\langle break, \sigma \rangle \mapsto_c \sigma[br/flag]} \qquad \frac{\sigma(flag) = ok \quad \langle a, \sigma \rangle \longrightarrow n}{\langle assign(X, a), \sigma \rangle \mapsto_c \sigma[n/X]} \\
\\
\frac{\sigma(flag) = ok \quad \langle c_1, \sigma \rangle \mapsto_c \sigma' \quad \sigma'(flag) = br}{\langle cseq(c_1, c_2), \sigma \rangle \mapsto_c \sigma'} \\
\\
\frac{\sigma(flag) = ok \quad \langle c_1, \sigma \rangle \mapsto_c \sigma'' \quad \sigma''(flag) = ok \quad \langle c_2, \sigma'' \rangle \mapsto_c \sigma'}{\langle cseq(c_1, c_2), \sigma \rangle \mapsto_c \sigma'} \\
\\
\frac{\langle b, \sigma \rangle \mapsto true \quad \sigma(flag) = ok \quad \langle c_1, \sigma \rangle \mapsto_c \sigma'}{\langle ifthenelse(b, c_1, c_2), \sigma \rangle \mapsto_c \sigma'} \\
\\
\frac{\langle b, \sigma \rangle \mapsto false \quad \sigma(flag) = ok \quad \langle c_2, \sigma \rangle \mapsto_c \sigma'}{\langle ifthenelse(b, c_1, c_2), \sigma \rangle \mapsto_c \sigma'} \\
\\
\frac{\sigma(flag) = ok \quad \langle c, \sigma \rangle \mapsto \sigma'' \quad \sigma''(flag) = ok \quad \langle repeat(c), \sigma'' \rangle \mapsto_c \sigma'}{\langle repeat(c), \sigma \rangle \mapsto_c \sigma'} \\
\\
\frac{\sigma(flag) = ok \quad \langle c, \sigma \rangle \mapsto_c \sigma' \quad \sigma'(flag) = br}{\langle repeat(c), \sigma \rangle \mapsto_c \sigma'[ok/flag]}
\end{array}$$

Il linguaggio FUN

Consideriamo un linguaggio di programmazione *funzionale*, che chiamiamo FUN. I programmi sono termini che contengono anche le espressioni aritmetiche e booleane. La sintassi è la seguente

$$\begin{aligned}
t ::= & x \mid n \mid true \mid false \mid t_1 + t_2 \mid t_1 - t_2 \mid t_1 \times t_2 \mid t_1 \wedge t_2 \mid t_1 \vee t_2 \mid \neg t_1 \mid \\
& t_1 = t_2 \mid t_1 < t_2 \mid if\ t_0\ then\ t_1\ else\ t_2 \mid fun(x, t) \mid apply(t_1, t_2)
\end{aligned}$$

dove n sono interi, $x \in Var$ sono le variabili, $true$ e $false$ sono le costanti booleane. Il predicato di uguaglianza è definito sugli interi, così come quello di minore.

La valutazione di un termine t avviene in uno stato (che spesso chiameremo ambiente) δ , che è una funzione da variabili Var ai valori, e restituisce un valore in un insieme $\mathcal{V}$. Tale insieme è il più piccolo insieme che contiene gli interi, i booleani e le astrazioni funzionali chiuse, le liste finite di valori e le coppie di valori. Un'astrazione funzionale chiusa è un oggetto della forma $fun(x, t)$ dove $FV(t) \subseteq \{x\}$.

L'insieme delle *free variables* del termine t , che indichiamo con $FV(t)$, è definito per induzione strutturare sui termini nel seguente modo:

$$FV(t) = \begin{cases} \{x\} & \text{se } t \text{ è } x \\ \emptyset & \text{se } t \text{ è } n, \text{true}, \text{false} \\ FV(t_1) \cup FV(t_2) & \text{se } t \text{ è } \begin{cases} t_1 + t_2, t_1 - t_2, t_1 \times t_2, t_1 \wedge t_2, t_1 \vee t_2, \\ t_1 < t_2, t_1 = t_2, \text{apply}(t_1, t_2) \end{cases} \\ FV(t') & \text{se } t \text{ è } \neg t' \\ FV(t') \setminus \{x\} & \text{se } t \text{ è } \text{fun}(x, t') \\ FV(t_0) \cup FV(t_1) \cup FV(t_2) & \text{se } t \text{ è } \text{if } t_0 \text{ then } t_1 \text{ else } t_2 \end{cases}$$

Quindi, indicando con $\mathbb{Z}$ gli interi, $\mathbb{B}$ i booleani e con $\mathcal{F}$ le chiusure, abbiamo che $\mathcal{V} = \mathbb{Z} \cup \mathbb{B} \cup \mathcal{F}$ e questo insieme è ben definito dato che i sottoinsiemi che lo compongono sono disgiunti a due a due.

La semantica del linguaggio è data dalle seguenti regole, dove δ è l'ambiente che è una funzione parziale che associa agli identificatori un valore in $\mathcal{V}$, e $\delta[\mathbf{v}/x]$ è la funzione ottenuta da δ così definita: $\delta[\mathbf{v}/x](x) = \mathbf{v}$ e $\delta[\mathbf{v}/x](y) = \delta(y)$.

$$\begin{array}{cccc} \overline{\langle x, \delta \rangle \rightarrow \delta(x)} & \overline{\langle n, \delta \rangle \rightarrow \mathbf{n}} & \overline{\langle \text{true}, \delta \rangle \rightarrow \mathbf{true}} & \overline{\langle \text{false}, \delta \rangle \rightarrow \mathbf{false}} \\ \\ \frac{\langle t, \delta \rangle \rightarrow \mathbf{v}}{\langle \neg t, \delta \rangle \rightarrow \neg \mathbf{v}} & \frac{\langle t_1, \delta \rangle \rightarrow \mathbf{v}_1 \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}_2}{\langle t_1 \text{ op } t_2, \delta \rangle \rightarrow \mathbf{v}_1 \text{ op } \mathbf{v}_2} & \mathbf{op}, \text{op} \in \{+, -, \times, <, =, \wedge, \vee\} & \end{array}$$

con $\mathbf{v}_1 \text{ op } \mathbf{v}_2$, dove $\mathbf{op} \in \{+, -, \times, <, =, \wedge, \vee\}$ intendiamo che $\mathbf{v}_1$ e $\mathbf{v}_2$ sono definiti e del *tipo* corretto e quindi anche $\mathbf{v}_1 \text{ op } \mathbf{v}_2$ è definito, mentre op è il corrispettivo nel linguaggio di programmazione.

$$\begin{array}{cc} \frac{\langle t_0, \delta \rangle \rightarrow \mathbf{true} \quad \langle t_1, \delta \rangle \rightarrow \mathbf{v}}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow \mathbf{v}} & \frac{\langle t_0, \delta \rangle \rightarrow \mathbf{false} \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow \mathbf{v}} \\ \\ \frac{FV(t) \subseteq \{x\}}{\langle \text{fun}(x, t), \delta \rangle \rightarrow \text{fun}(x, t)} & \frac{\langle t_1, \delta \rangle \rightarrow \text{fun}(x, t) \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}_2 \quad \langle t, \delta'[x/\mathbf{v}_2] \rangle \rightarrow \mathbf{v}}{\langle \text{apply}(t_1, t_2), \delta \rangle \rightarrow \mathbf{v}} \end{array}$$